МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине «Построение и анализ алгоритмов»

Тема: Поиск подстроки в строке

Студентка гр. 4345 _____ Левкович Ю.С.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2025

Цель работы.

Изучить и реализовать алгоритм Кнута-Морриса-Пратта для поиска индексов вхождения подстроки в строку и проверки строки на циклический сдвиг.

Задание

Часть 1.

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T.

Вход:

- Первая строка — P

- Вторая строка — T

Выход:

индексы начал вхождений P в T, разделённые запятой; если P не входит в T, то вывести -1.

Sample Input:

ab

abab

Sample Output:

0,2

Часть 2.

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B, склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A

- Вторая строка — B

Выход:

Если A является циклическим сдвигом B, то индекс начала строки B в A; иначе вывести -1.

Если возможно несколько сдвигов, вывести первый индекс.

Sample Input:

defabc

abcdef

Sample Output:

3

Выполнение работы:

В ходе выполнения работы были реализованы две функции. prefix_func(P) отвечает за вычисления префикс функции для переданного шаблона P. В ней создаётся массив pref, в котором будут храниться длины префиксов. Для его заполнения пройдём по каждому символу шаблона и будем изменять величину последнего записанного совпадения k = pref[i - 1], если символы совпадают увеличиваем совпадение и записываем в его в массив pref[i] = k, если символы не совпадают, то берём предыдущие совпадения, пока не будут найдены первые одинаковые (если таких нет, будет записан 0).

Основной цикл работы алгоритма был реализован в функции KMP(A, B). В первую очередь в ней вызывается функция prefix_func и создаётся массив, в который будут записываться результаты. Далее создаются два счётчика: счётчик i идёт по тексу, а счётчик k по шаблону (и префикс функции). По умолчанию k = 0, если символ в тексте на позиции i совпадает с символом в шаблоне, то k увеличивается на 1 и проверяются следующие символы, если это не так, то k приравнивается к значению префикс функции для символа, стоящего на позиции k – 1, пока не будет найдено совпадение. Если в результате работы был проверен весь шаблон (т.е. k стало равным длине P) в результат добавляется индекс начала совпадения (из текущего i вычитается длина шаблона), а k уменьшаем до следующего возможного совпадения.

Вторая часть почти полностью повторяет логику оригинального алгоритма, за исключением того, что за T принимается удвоенная входная

строка, тогда, если в исходное предложение является сдвигом шаблона, в новом тексте будет подстрока, совпадающая с ним. Для нахождения сдвига нужно запустить алгоритм и вывести индекс вхождения шаблона, это и будет результат работы.

Временная сложность работы алгоритма составляет O(n + m), она получается из временных сложностей двух основных функций: prefix_func (O(m)) и KMP (O(n)).

Оценка сложности алгоритма по памяти показывает, что она может быть в худшем случае O(m + n), O(m) используется для хранения префикс функции, а O(n) для хранения результата (если каждый символ является шаблоном).

Исходный код программы смотри в Приложении 1.

№	Входные данные	Выходные данные	Комментарий
1	aba abacbaba	0,5	Задание 1. Поиск вхождений подстроки в строку. 2 Вхождения.
2	kjk asaopaflo	-1	Задание 1. Поиск вхождений подстроки в строку. 0 вхождений.
3	a aaaaaaaa	0,1,2,3,4,5,6,7	Задание 1. Поиск вхождений подстроки в строку. Все символы в строке являются шаблоном.
4	qwerty rtyqwe	3	Задание 2. Проверка на циклический сдвиг. Он есть.
5	hdgahg asowri	-1	Задание 2. Проверка на циклический сдвиг. Его нет.
6	qwerty rtyqwerty	-1	Задание 2. Проверка на циклический сдвиг. Шаблон является подстрокой текста, но не является циклическим сдвигом.

Таблица 1 – тестирование работы программы.

Вывод.

Был изучен и реализован алгоритм Кнута-Морриса-Пратта для поиска подстроки в строке. А также была добавлена модификация для вычисления циклического сдвига.

ПРИЛОЖЕНИЕ 1

ИСХОДНЫЙ КОД ПРОГРАММЫ

Задание 1.

```
def prefix_func(P):
    pref = [0] * len(P)
    for i in range(1, len(P)):
        k = pref[i - 1]
        while k > 0 and P[k] != P[i]:
            k = pref[k - 1]
        if P[k] == P[i]:
            k += 1
        pref[i] = k
    return pref

def KMP(T, P):
    k = 0
    A = []
    pref = prefix_func(P)
    for i in range(len(T)):
        while k > 0 and T[i] != P[k]:
            k = pref[k - 1]
        if T[i] == P[k]:
            k = k + 1
        if k == len(P):
            A.append(i - len(P) + 1)
            k = pref[k - 1]
    return A

P = input()
T = input()
res = KMP(T, P)
if res != []:
    res = list(map(str, res))
    print(",".join(res))
else:
    print(-1)
```

Задание 2.

```
def prefix_func(P):
    pref = [0] * len(P)
    for i in range(1, len(P)):
        k = pref[i - 1]
        while k > 0 and P[k] != P[i]:
            k = pref[k - 1]
        if P[k] == P[i]:
            k += 1
        pref[i] = k
    return pref

def KMP(A, B):
    if len(A) != len(B):
        return -1
    T = B + B
    P = A
```

```
    k = 0
    pref = prefix_func(P)
    for i in range(len(T)):
        while k > 0 and T[i] != P[k]:
            k = pref[k - 1]
        if T[i] == P[k]:
            k = k + 1
        if k == len(P):
            return i - len(P) + 1
    return -1

P = input()
T = input()
print(KMP(T, P))
```